



Experiment 8.2

Student Name: Niharika Singh

Branch: BE-CSE

Semester: 5th

UID: 23BCS11624

Date of upload: 27 OCT 2025

Subject Name: FS

1. Aim: implement Protected Routes with JWT Verification

2. Objective: Learn how to secure backend API routes using JSON Web Tokens (JWT) to ensure that only authenticated users can access certain resources. This task helps you understand token-based authentication, how to verify JWTs on the server side, and how to protect specific routes from unauthorized access.

4. Implementation/Code:

```
const express = require('express')
const jwt = require('jsonwebtoken')
const bodyParser = require('body-parser')
const cors = require('cors')

const app = express()
app.use(cors())
app.use(bodyParser.json())

// Secret key for signing JWTs
const JWT_SECRET = 'mysecretkey123'
```

```
// Sample hardcoded user
const user = {
  username: 'admin',
  password: '12345'
}

// Login route - generates a JWT token
app.post('/login', (req, res) => {
  const { username, password } = req.body

  if (username === user.username && password === user.password) {
    const token = jwt.sign({ username }, JWT_SECRET, { expiresIn: '1h' })
    return res.json({ success: true, token })
  }

  return res.status(401).json({ success: false, message: 'Invalid credentials' })
})

// Middleware to verify JWT
function verifyToken(req, res, next) {
  const authHeader = req.headers['authorization']
  const token = authHeader && authHeader.split(' ')[1]

  if (!token) return res.status(403).json({ message: 'No token provided' })

  jwt.verify(token, JWT_SECRET, (err, decoded) => {
    if (err) return res.status(401).json({ message: 'Invalid token' })
    req.user = decoded
    next()
  })
}
```

```

    })
  }

// Protected route
app.get('/protected', verifyToken, (req, res) => {
  res.json({
    message: 'Access granted to protected route',
    user: req.user
  })
})

// Public route
app.get('/', (req, res) => res.send('JWT Authentication API is running'))

const PORT = 4000
app.listen(PORT, () => console.log(`Server running on port ${PORT}`))

```

5. Output:

JWT Protected Route Demo

Username

Password

Login

Access Protected

Response:

No response yet

Learning Outcome :

You will learn how to use React's useState hook to manage and validate form data, handle user input in real time, and respond to form submission events effectively within a component.